Code Jam 2011 - World Finals

Analysis: Ace in the Hole

This is a rather unconventional problem that requires a lot of thought up-front and relatively little implementation.

Let's think about the problem as a competitive game. Ben chooses a card position, and then Amy chooses the value of that card. Amy is not allowed (a) to re-use values, (b) to create a decreasing subsequence of length 3, or (c) to place the 1 before the final turn. Our task it to understand what decisions Amy could have made during the game. We will present the answer first, and then prove it is correct. Define an "adversarial strategy" for Amy as follows:

- At any point, consider the cards that Ben has not yet looked at. Suppose they are in positions $p_1 < p_2 < ... < p_m$, and have values $v_1 < v_2 < ... < v_m$.
    - If Ben looks at the card in position p_k with $k < m$, then Amy assigns it value v_{k+1}.
    - If Ben looks at the card in position p_m and either (a) $m \leq 2$, or (b) there is a previously revealed card in position less than p_m with value between v_{m-1} and v_m, then Amy assigns it value v_m. Otherwise, she assigns it value *either* v_{m-1} or v_m.

We claim that the problem conditions are equivalent to saying that Amy chooses the deck values according to an adversarial strategy. Once that is established, it will be pretty straightforward to find the lexicographically largest solution.

Adversarial Strategies Cannot Be Exploited

The main technical challenge is to prove that adversarial strategies really do require Ben to look at every card. On a contest of course, you would not need to make this argument quite so rigorously.

Lemma: If the deck values are assigned according to an adversarial strategy, then the deck contains no decreasing subsequence of length 3.

Proof: We follow along as Ben looks at the cards one at a time. At each step, we will say a card is "safe" if Ben has looked at it, and either (a) all cards with a higher position have also been looked at, or (b) all cards with a higher value have also been looked at. Let's suppose the remaining cards are in positions $p_1 < p_2 < ... < p_m$, and have values $v_1 < v_2 < ... < v_m$. These are the "unsafe" cards.

We claim the following is true:

- There is no way of assigning values to the remaining cards that will make a decreasing subsequence of length 3 with a safe card.
- If an unsafe card in position p_i has been looked at, then it has value v_{i+1}.

We prove this claim by induction on the number of cards that Ben has looked at. Initially, all cards are unsafe and the claim obvious.

Now let's suppose Ben looks at a card *C* in some position p_k.

- *Case 1:* k < m - 1. Since Ben has not looked at the card in position p_m (or else it would be safe), the value of card *C* will be set according to the first adversarial strategy rule. Specifically, if there are q cards in positions less than p_k that have not been looked at, then *C* will be assigned the $(q+1)^{th}$ smallest unrevealed value. As all safe cards have been revealed, we can restrict our attention to unsafe cards, from which the inductive hypothesis makes it clear that the $(q+1)^{th}$ smallest unrevealed value is v_{k+1}. Since k + 1 < m, the set of safe cards will not change because of this step, and the inductive hypothesis will still be satisfied afterwards.

- *Case 2:* k = m - 1. By the same reasoning as in Case 1, *C* will be assigned value v_m. However, the set of safe cards will change in this case. Suppose Ben has already looked at the cards in positions p_u, p_{u+1}, ..., p_{m-2}, but he has not looked at the card in position p_{u-1}. These cards will all be labeled as safe set because they have high values, while all other cards will remain unsafe. We need to show that they cannot be part of a decreasing subsequence of length 3. By the inductive hypothesis, we can ignore cards that had been previously marked as safe for this purpose. Now, the newly labeled safe cards are arranged in increasing order, there is only one unsafe card positioned after them, and no preceding unsafe card can have higher value, so indeed, they cannot be part of a decreasing subsequence of length 3. Therefore, the inductive hypothesis is once again satisfied.

- *Case 3:* k = m. Suppose *C* is assigned value v_t. We know Ben has not looked at the card with value v_m, or else that card would already be safe. If Ben has also looked at the card with value v_{m-1}, then the adversarial condition forbids *C* from having value less than v_{m-1}. Otherwise, v_{m-1} and v_m are both still unrevealed, and the adversarial condition demands that *C* have value v_{m-1} or v_m.
    - If t = m-1, then *C* will be marked as safe but no other cards will. (Ben cannot have looked at the card in position p_{m-1} because it would have value v_m, and would therefore be safe already.) Furthermore, it cannot be part of a length-3 decreasing subsequence because no unsafe card has a larger position and at most one unsafe card can have higher value.
    - If t = m, then some additional cards p_u, p_{u+1}, ..., p_{m-2} might also get marked safe. The newly labeled safe cards are arranged in increasing order, there is only one unsafe card positioned after any of them, and no preceding unsafe card can have higher value, so no decreasing subsequence of length 3 can include these cards. Therefore, the inductive hypothesis is once again satisfied.

In any case, the inductive hypothesis holds at each step, and therefore the lemma is proven.

It now follows immediately that Ben requires all N guesses if the cards are assigned according to an adversarial strategy. No matter what cards Ben looks at, Amy can continue her adversarial strategy and avoid revealing either a decreasing subsequence of length 3 or the card with value 1. In particular, Ben can never finish early.

Non-Adversarial Orders Can Be Exploited

The previous argument is important from a technical perspective, but in practice, you would begin solving this problem from the other side: first showing how Ben can exploit poor strategies. Towards that end, we show that if Amy ever deviates from an adversarial strategy, then Ben can find the 1-card without requiring all N turns.

Let's begin by focusing on the first card Ben looks at.

Observation 1: Suppose Ben looks at card $i < N$. Then Amy must assign it value $i + 1$.

Reason: Suppose he sees value $j \neq i + 1$. We claim that the value-1 card cannot be in position N. Indeed, if it was in that position, then the condition that the deck has no decreasing subsequence of length 3 implies that (a) all cards with position less than i have value in the range [2, j-1], and (b) all cards with position between $i + 1$ and $N - 1$ have value in the range [j+1, N]. If $j < i + 1$, then there are not enough cards in the range [2, j-1] for this to be possible, and if $j > i + 1$, then there are not enough cards in the range [j+1, N] to be possible.

Therefore, if Amy assigns this card a value other than $i + 1$, then Ben need never look at card N. But we know he did indeed have to look at all the cards, so it must be that Amy assigned value $i + 1$.

Observation 2: Suppose Ben looks at card N. Then Amy must assign it value $N - 1$ or N.

Reason: Suppose he sees value $j < N - 1$. If $N = 3$, then Ben just found the value-1 card, which is an immediate contradiction. Otherwise, $N \geq 4$, and we show that Ben can find the 1-value card in less than $N - 1$ further card checks, which is another contradiction.

Forgetting the information Ben has already obtained, we know the remaining $N - 1$ card values are placed in the $N - 1$ preceding positions. By Observation 1, if Ben checks the card in position i, then it must have the $(i+1)^{th}$ smallest value of the remaining cards. (Otherwise, we already know he can find the 1-value card without checking everything.) So if Ben checks the cards in all positions except $N - 3$ and $N - 1$, he must see the following values:

2, 3, ..., j-1, j+1, j+2, ..., N-2, ???, N, ???, j.

The two unknown cards have value 1 and $N - 1$ in some order. However, we know the second-last card cannot have value $N - 1$, or we would have a decreasing subsequence: N, $N - 1$, j. Therefore, the 1 must be in that position, and so Ben never needs to check the fourth-last card, and the proof is complete.

There is only one tricky case left. At a given point, let the remaining unrevealed cards be in positions $p_1 < p_2 < ... < p_m$, and have values $v_1 < v_2 < ... < v_m$. Suppose that there is a previously revealed card in position less than p_m with value between v_{m-1} and v_m. We need to show that if Ben looks at the card in position p_m, then Amy must assign it value v_m.

Reason: Suppose Amy does not do this. Consider the first time where she does not. Let *C* be the card in position p_m, and let *C′* be the revealed card that is positioned before *C* and that has value between v_{m-1} and v_m.

Up until this point, Amy has followed an adversarial strategy, so we can use the inductive statement proven in our first lemma to divide the deck into safe and unsafe cards. We know there is an unrevealed card positioned after *C′* (namely *C*) and there is an unrevealed card value that is higher than the value of *C′* (namely v_m), so *C′* must be an unsafe card.

Let the unsafe cards have positions $q_1, q_2, ..., q_r$ and values $w_1, w_2, ..., w_r$. Since v_{m-1} is an unrevealed value, it belongs to an unsafe card and we have $v_{m-1} = w_u$ for some u. Also *C′* has

value w_t for some t > u and position p_{t-1}.

Now suppose Amy assigns *C* a value of $v_{m-1} = w_u$. Then the card in position q_{u-1} must be unrevealed, or it would have had the same value. Also, as argued earlier, the card in position q_{r-1} must be unrevealed or it would have value w_r and therefore be safe already. Ben can now exploit Amy's strategy by looking at every card other than q_{u-1} and q_{r-1}. It is easy to check this will leave only the values 1 and w_{u+1}. But the card in position q_{r-1} cannot have value w_{u+1} or we would get a decreasing subsequence: w_t, w_{u+1}, w_u. Therefore, this card must have value 1, and Ben can avoid checking the card with position q_{u-1}. This proves the final part!

Finding The Lexicographically Largest Solution

We are now finally ready to discuss the solution. At each step, Amy's strategy is almost completely determined. The only question is whether she should assign v_m or v_{m-1} to the card in position p_m when she has the choice. In fact, the two choices lead to identical deck orderings except with v_m and v_{m-1} swapped. In order to make the ordering as large lexicographically as possible, we want v_m early on, and so we should always prefer using v_{m-1}.

That's it! We just need to implement the adversarial strategy, always preferring v_{m-1} over v_m. But of course it took a lot of reasoning to get to this stage!

Code Jam 2011 - World Finals

Analysis: Google Royale

Google Royale is perhaps the most unapproachable problem we have ever posed for the Google Code Jam. It requires a great deal of understanding before any progress at all can be made on the large input. This is because the interaction between winning probability and starting money is rather complicated, and you cannot possibly afford to do a complete search through all 10^{16} states. Although the scenario here might seem similar to Millionaire, a previous Code Jam problem, you simply cannot afford the kind of computation that worked there.

First, let's try to understand a betting round. Let the initial bet be **y**. If you lose k times and then win, your total earnings will be **y** * 2^{k-1} - **y** * 2^{k-2} - **y** * 2^{k-3} - ... - **y**, which equals **y** no matter what k is. If you lose, your total earnings will be negative and the exact value will depend on how many times you doubled.

The key to the whole problem is that your *expected* winnings from a betting round is exactly 0, no matter what your initial bet is and no matter how many times you are willing to double if you keep losing. (Recall that the *expected* winnings is your "average" winnings -- formally it is defined as sum p_i * i, where p_i is the probability that you win exactly i dollars.) The expected value is 0 because at each step of each round, you have a 50% chance of winning some money, and a 50% chance of losing the same amount of money. The average winnings is always 0 in each step, and therefore 0 overall.

Now let's think about strategy a little bit. In general, there will be several different strategies that maximize your probability of winning. For now, we will work on identifying only one of them. We will come back to the question of finding the maximum bet later.

Observation 1: If you have **x** dollars, there is no reason to start a betting round with more than **V** - **x**.

Reason: If you bet more than **V** - **x**, then winning will always put you over **V**. If you decrease the bet slightly, then winning is just as good, but losing is no worse. Therefore, you might as well bet less.

From now on, we will always restrict our attention to strategies that follow Observation 1, and therefore, we will never end up with *more* than **V** dollars, no matter how lucky or unlucky we are.

Observation 2: Fix a strategy. Let **P** be the probability of reaching **V** dollars, and let **L** be the expected number of dollars you end up with if you lose. Then **P =** 1 - (**V** - **A**) / (**A** - **L**). Since **V** and **A** are fixed, this means maximizing **P** is equivalent to minimizing **L**.

Reason: This follows from the fact that your expected winnings after any number of betting rounds is always equal to 0, or equivalently, your total expected money is always equal to **A**. Let's see what this tells us at the end of all betting rounds. At that point, you will have won with probability **P**, in which case your money is exactly **V** by Observation 1. Or you will have lost with probability 1 - **P**, in which case your expected money is exactly **L**. Therefore, we get:
P * **V** + (1 - **P**) * **L = A**

=> **P** * (**V** - **L**) = **A** - **L**
=> **P** = 1 - (**V** - **A**) / (**A** - **L**).

So now the question reduces to making **L** as small as possible. Using these ideas, we can make a couple major deductions about the optimal strategy.

Observation 3: If you have **x** dollars, you might as well do one of two things: (1) bet exactly **x** and double until you win or until it is no longer possible to double, or (2) bet exactly 1 and do not double even if you lose.

Reason: An optimal strategy will bet some number **y** and double up to **k** times. If one of the betting steps results in a win, you will end up with **x** + **y** dollars. Otherwise, you will end up with some number **x** - **z** dollars. As always, your expected number of dollars is constant, and so is equal to **x**. We will show how to replace the strategy of betting **y** and doubling up to **k** times with a new strategy (possibly requiring multiple betting rounds) that follows the rules we want, and that is at least as effective at reaching **x** + **y** dollars.

Case 1: **x** - **z** ≥ 0. Instead of betting **y**, repeatedly bet 1 and do not double. Stop only when your total amount of money increases to **x** + **y** or decreases to **x** - **z**. Since **x** - **z** ≥ 0, it is legal to continue betting until one of these outcomes happens. Like the original strategy, this strategy will result in the same two outcomes (**x** + **y** dollars or **x** - **z** dollars), and the expected money at the end will still be **x** for the exact same reason. However, as Observation 2 shows, if two strategies have identical winning and losing outcomes and identical expected values at the end, they have identical probability of getting the better outcome. Therefore, this strategy is identical to the original strategy, and we can just do this instead.

Case 2: **x** - **z** < 0. Instead of betting **y**, repeatedly bet 1 and do not double. Stop only when your total amount of money increases to **x** + **y** or decreases to **y**. If you end up at **y**, then bet **y**, and keep doubling until you win or go broke. If you win, go back to the first step and continue betting 1 until you reach **x** + **y** or return back down to **y**. If you end up at **y** again, bet **y**, and repeat. Like the original strategy, this will end with you having exactly **x** + **y** dollars or with you being broke. However, it is easy to check that **L** is strictly smaller here than in the original strategy while the expected money at the end, as always, stays equal to **x**. Therefore Observation 2 guarantees that this new strategy is strictly better than the original strategy.

We have shown that any strategy can switch to betting 1 or **x** without becoming any worse, and that proves Observation 3.

This is a very powerful observation, but there is still more to understand! With 10^{16} possible money amounts, we cannot afford to try both strategies in every situation. There is one more idea that drastically cuts down the search space.

Observation 4: For each non-negative integer i, let M_i be the largest integer such that $M_i * 2^i \leq$ **M**. We will call these numbers "inflection points". Then you should never bet all your money unless the amount you have is an inflection point.

Reason: The proof here is very similar to Observation 3. Consider a strategy that bets everything for **x** satisfying $M_i < \mathbf{x} < M_{i-1}$. Let **y** equal **x**/2, rounded up. Note that $2\mathbf{y} \leq \mathbf{x} + 1 \leq M_{i-1}$, so if we bet **y**, then we can double i times.

The current strategy either gives you 2**x** dollars or **x** * (1 - 1 - 2 - ... - 2^{i-1}) = -2**x** * (2^{i-1} - 1). However, **y** / **x** ≥ 1/2 > (2^{i-1} - 1) / (2^i - 1). Therefore, the losing amount of money for this strategy is more than -2**y** * (2^i - 1).

Now we consider an alternate strategy. Instead of going all in at **x**, repeatedly bet 1 and do not double. Stop only when your total amount of money increases to 2**x** or decreases to **y**. In the latter case, go all in and double up to i times, or until you win. As noted above, your bet will never exceed **M**, so this is a legal strategy. If you win, start over from the beginning. Like the original strategy, this will either leave you with 2**x** dollars or broke. However, if you are broke, you end up with **y** * (1 - 1 - 2 - ... - 2^i) = -2**y** * (2^i - 1). As argued above, this losing value is less than it was for the original strategy, and so Observation 2 implies the new strategy has a higher probability of reaching 2**x**. Therefore, the original strategy could not have been optimal.

When to go All-In: Let's say a dollar amount is an "all-in" point if we should bet everything when we have that amount of money. Observation 4 guarantees that all-in points are a subset of the inflection points, but it is not true that every inflection point is an all-in point.

Consider the inflection points in increasing order. The smallest inflection point will be 1, and certainly that is an all-in point. Let's now consider the second smallest inflection point. Observation 2 says that our goal is to minimize **L**, so we calculate what we end up with if we go all in from this inflection point and lose. If it is our lowest total yet, we *should* go all in there. Otherwise, we can do better by betting 1 until we get to the lower inflection point. The same argument applies for the third inflection point, and then the fourth and so on. In this way, we can very quickly calculate an optimal strategy at every dollar amount.

There are two tricky cases to watch out for here:

- By Observation 1, we should only consider inflection points that are at most **V** / 2.
- It might be that going all-in at a point x is equally effective as betting 1 and waiting until the next all-in point. In this case, we will say x is an "optional all-in point". The other all-in points are called "strict all-in points".

Calculating the Winning Probability: Let P_x denote the probability of reaching **V** if your current amount of money is x. If x is not a strict all-in point, then one optimal strategy is to bet 1 until we reach the strict all-in point directly above x or directly below x, or until we reach **V** itself. For any x between these all-in points, we have the recurrence: $P_x = (P_{x-1} + P_{x+1})/2$, or in other words, $P_x - P_{x-1} = P_{x+1} - P_x$. This implies P_x is linear in this range, and therefore we can calculate P_x if we know the value of P at both endpoints.

We can now calculate P by starting from the largest all-in point and working down. For example, suppose the largest all-in point is y and the probability of losing immediately from going all in there is 1 - p. Then $P_y = p * P_{2y}$. Also, by linearity, $P_{2y} = P_y$ * (**V** - 2y) / (**V** - y) + 1 * y / (**V** - 2y). We know p, **V**, and y, so it is easy to calculate P_y from here. Once we have P_y, we can use the same trick to calculate P for the second-largest all-in point, then for the third-largest all-in point, and so on, until eventually we have calculated all probabilities.

Calculating the Largest Optimal Bet: It remains only to calculate the largest optimal bet. If **A** is an all-in point, either strict or optional, then certainly we can and should bet **A** there.

Otherwise, consider a dollar amount x. If x is not a strict all-in point, then we already saw that $P_x - P_{x-1} = P_{x+1} - P_x$. This means P_x is completely linear between strict all-in points, and so we can certainly increase the bet until either winning or losing would take us to a strict all-in point on either side. If x is equal to a strict all-in point however, then 1 is not an optimal bet, and we have $P_x > (P_{x-1} + P_{x+1}) / 2$, or equivalently, $P_x - P_{x-1} > P_{x+1} - P_x$. In particular, this means that increasing the bet further would hurt us. Therefore, the largest possible bet is the distance between **A** and the nearest strict all-in point.

Code Jam 2011 - World Finals

Analysis: Program within a Program

Here at the Google Code Jam, we have always tried to encourage a variety of programming languages. But in this problem, we took things a little further by forcing you to program in the language that started them all -- the abstract Turing Machine! Unfortunately, abstract Turing Machines are pretty unwieldy, and that makes even the simple task here quite difficult.

With so few rules allowed, it is important to take advantage of the numbers you can mark onto the lampposts. The obvious approach is to write down, perhaps in binary, the number of steps you need to make, and then have the robot makes decisions based on that. Unfortunately, there is also a rather tight limit on the number of moves, which means you cannot afford to keep going back to the start to check on your number. You will need to take the data with you as you move.

Here is one effective way of doing this:

- First write down the distance you want to go forward in binary, using the values 1 and 2 to represent the binary digits. This information will now be available on the starting lampposts.
- Now repeat the following:
  - Trace through the number from right to left, and subtract 1 as you go.
  - If the number was 0, then drop the cake right now.
  - Otherwise trace through the number from left to right, copying everything one position to the right.

Once you have the high level algorithm, each piece is relatively straightforward to implement. For example, subtracting 1 comes down to formalizing the subtraction algorithm you learned in grade school:

- Start in state 1, which we'll you use to mean you have not yet done the subtraction. If the last digit is 1, you can replace it with 0 and the subtraction is done, so switch to state 2. If the last digit is 0, replace it with 1 but you now need to borrow 1 from the previous digit, so stay in state 1. Either way, move to the previous digit.
- Once you are in state 2, you are just moving left through the number without changing anything.
- If you reach the left end of the number in state 1, then the number must have been 0, so you should drop the cake. Otherwise, you should move onto the copy phase.

And the copy phase is similar but conceptually simpler.

Almost any implementation of this algorithm should be good enough to solve the problem, but there is room for more optimization if you want a challenge! With the 30-rule limit, we were able to bring the number of steps down to about 95,000. Here is the full program to move 5,000 lampposts:

```
0 0 -> E 1 1
1 0 -> E 2 3
2 0 -> E 3 1
3 0 -> E 4 3
4 0 -> E 5 2
5 0 -> E 6 3
6 0 -> E 7 1
7 0 -> E 8 3
8 0 -> W 9 3
```

```
9 0 -> R
10 0 -> E 11 0
9 1 -> W 9 3
10 1 -> W 10 1
9 2 -> W 10 1
10 2 -> W 10 2
9 3 -> W 10 2
10 3 -> W 10 3
11 1 -> E 12 0
12 0 -> W 9 3
12 1 -> E 12 1
12 2 -> E 13 1
12 3 -> E 14 1
13 0 -> W 10 1
13 1 -> E 12 2
13 2 -> E 13 2
13 3 -> E 14 2
14 0 -> W 10 2
14 1 -> E 12 3
14 2 -> E 13 3
14 3 -> E 14 3
```

Can you do better? We'd love to hear about it if so!

Code Jam 2011 - World Finals

Analysis: Rains Over Atlantis

One obvious solution (which works for small input) is to simulate. The trouble starts when the heights are large, and **M** is small. Thus, we need to be able to do multiple steps at once.

Begin by noting that, given the current map of heights, we can determine the water levels (i.e., what areas are submerged and what areas are not) using a single run of Dijkstra's shortest path algorithm, in RC log(RC) time.

We will have a single step procedure. What it does is:

1. Calculate water levels for all fields.
2. For each field find out if it is submerged; if not, find the lowest water level in the adjacent fields.
3. Create a new map in which all the heights are lessened by the appropriate erosion, and increase the day counter by one.

This is one step of the simulation, which is also needed for the naive solution.

We will also have a multistep procedure, which tries to perform multiple single steps at once, as long as all of the following are true:

- all non-submerged fields erode at maximum speed,
- the water levels of all submerged fields decrease at the same speed, and
- no submerged field becomes un-submerged during these steps.

The multistep procedure will work as follows:

1. Calculate the water levels for all fields.
2. For each field find out whether it is submerged. If not, find out how much it will erode. If this is not **M**, break the multistep. Note that we can choose to erode fields to below sea level for simplicity without changing the result.
3. If all fields erode at speed **M**, then the water levels of all "lakes" also decrease at this speed. Note that each lake has at least one field on the border which is not submerged, but determines the water level of the lake, and this level will decrease by **M**. Thus, we can continue repeating steps until some field that was submerged surfaces.
4. Thus, we find the submerged field with the smallest amount of water on top, and batch the appropriate number of steps together.
5. If there are no submerged fields, then we increment the day counter by ceil(maxheight / M) and finish the algorithm.

Note that in each multistep one field that was submerged becomes uncovered, and it's easy to see that a field that is not submerged will never become submerged in the future - thus there are at most RC multisteps in the algorithm (actually, fewer because the fields on the edge of the board are never submerged).

Now we want to know how many steps are possible before we can perform a multistep. Define an auxilliary graph that has a node for each un-submerged field. For each lake we join all the fields in this lake to an arbitrary field on the boundary of the lake which defines the water level for the lake (the outlet of the lake). We define natural edges in the graph -- two nodes have an edge if any of the two fields merged into the two nodes were adjacent. For each nodes we can define its "parent" to be one of the nodes with the lowest level adjacent to it. Thus, we have a tree, rooted in the external sea. We define any path going upward in the tree to be "fixed" if all of the edges along the path have a height difference of at least **M**.

We can now prove that each day either

- a submerged becomes uncovered (thus decreasing the number of multisteps available), or
- the number of vertices that have fixed paths to the root increases, or
- all vertices lie on fixed paths.

If all vertices lie on fixed paths, then we can perform a multistep. As there can be at most RC increases before all vertices are on fixed paths, this means we get at most RC steps and at most one multistep before some field is uncovered, meaning at most $(RC)^2$ steps and RC multisteps in total, giving a running time of $(RC)^3 \log(RC)$. The constants are very small here, so this will easily run in time.

Now consider any fixed path. We assume the node distribution remains the same (i.e. the lakes remain lakes, and nothing gets uncovered). First notice that it remains a fixed path after one day - all of the fields on the path decrease by the same amount, so the differences remain the same. Now take any vertex that is not on a fixed path, but its parent is. Note that the sea level is on a fixed path by definition, so if no such vertex exists, it means that all vertices are on a fixed path. Then after one day the height of this vertex decreases to the height of the parent, while the height of the parent decreases by **M**. So the new difference is **M**, meaning that the vertex has joined the fixed path. This ends the proof.

Note that the upper bound can actually be (more or less) hit in the following map with M = 2: take a winding path of length that is on the order of RC, that does not touch itself, like so:

X X X X X X X
P P P P P P X
X X X X X P X
X P P P P P X
X P X X X X X
X P P P P P X
X X X X X X X

Where X represent very large values, and P represents the path. Thus, for the foreseeable future, the tree described above is actually the single path marked by Ps.

Now take the following values on the path: A, A-2L-1, A+1, A - 4L - 1, A+1, A - 6L - 1, A+1, A - 8L - 1, etc., where L is the length of the path. It is not difficult to check that "off by one" changes get propagated up the path with speed one, and each change gets to be fully propagated before the next one kicks in (due to the uncovering of another lake).

Code Jam 2011 - World Finals

Analysis: Runs

On a contest made up of unconventional problems, this one stands out as being a little more "normal" than the rest. That doesn't make it easy though! The main techniques here are good old-fashioned dynamic programming and counting, but you need to be very good at them to stand a chance.

The most important observation is that you cannot afford to build strings from left to right. With 450,000 characters, there is just far too much state to keep track of. The correct approach turns out to be placing all of one character type, then all of the next, and so on. Towards that end, let's define N_c to be the number of characters of type c, and define $X_{c,r}$ to be the number of ways of arranging all characters of type 1, 2, ..., c so that there are exactly r runs. We will compute the X values iteratively using dynamic programming.

Consider a string S using only the first c - 1 character types, and having r_0 runs. Note that S has exactly $M = N_1 + N_2 + ... + N_{c-1}$ characters altogether. Additionally, it has a few different types of "character boundaries":

- There is the boundary before the first character and the boundary after the last character. If we add a run of type-c characters in either one of these locations, the total number of runs will increase by 1.
- There are $r_0 - 1$ boundaries between distinct characters. If we add a run of type-c characters in any one of these locations, the total number of runs will increase by 1.
- There are $M - r_0$ boundaries between identical characters. If we add a run of type-c characters in any one of these locations, the total number of runs will increase by 2.

So let's suppose we add x runs of type-c characters in any one of the $r_0 + 1$ boundaries from the first two groups, and we add y runs of type-c characters in any of the $M - r_0$ boundaries from the the third group. There are exactly ($r_0 + 1$ choose x) * ($M - r_0$ choose y) ways of choosing these locations, and we will end up with $r_0 + x + 2y$ runs this way. Finally, we need to divide up the N_c characters of type c into these x + y runs. This can be done in exactly ($N_c - 1$ choose x + y - 1) ways. To see why, imagine placing all the runs together. Then we need to choose x + y - 1 run boundaries from $N_c - 1$ possible locations. See here for more information.

Therefore, the number of ways of adding all N_c type-c characters to S so as to get a string with exactly r runs can be calculated as follows:

- Loop over all non-negative integers x, y such that $r_0 + x + 2y = r$.
- Add ($r_0 + 1$ choose x) * ($M - r_0$ choose y) * ($N_c - 1$ choose x + y - 1) to a running total.
- After looping over all x, y, this running total will contain the answer we want.

Note that the answer here depends only on r_0. Therefore, the total contribution from all strings with r_0 runs is exactly $X_{c-1,r0}$ times this quantity. Iterating over all r_0 gives us the recurrence we need for X!

This method is actually quite fast. We can use O(450,000 * 100) time to pre-compute all the choose values. Everything else runs in $O(26 * 100^3)$ time.

By the way, you probably need to calculate the choose values iteratively rather than recursively. 450,000 recursive calls will cause most programs to run out of stack space and crash! Here is a

pseudo-code with a sample implementation (modulo operations removed for clarity):

```
def CountTransitions(M, Nc, r0, r):
  # Special case: If adding the first batch of characters the
  # only possible result is to have one run, and there is only
  # one way to achieve that.
  if r0 == 0:
    return r == 1 ? 1 : 0
  result = 0
  dr = r - r0
  for (y = 0; r0 + 2 * y <= r; ++y):
    x = r - (r0 + 2 * y)
    nways_select_x = Choose(r0 + 1, x)
    nways_select_y = Choose(M - r0, y)
    nways_split = Choose(Nc - 1, x + y - 1)
    result += nways_select_x * nways_select_y * nways_split
  return result


def Solve(freq, runs_goal):
  runs_count = [0 for i in range(0, runs_goal + 1)]
  runs_count[0] = 1
  M = 0
  for i, Nc in enumerate(freq):
    if Nc > 0:
      old_runs_count = list(runs_count)
      runs_count = [0 for i in range(0, runs_goal + 1)]
      for (r0 = 0; r0 <= runs_goal; ++r0):
        for (int r = r0 + 1; r <= runs_goal; ++r):
          nways = CountTransitions(M, Nc, r0, r)
          runs_count[r] += nways * old_runs_count[r0]
      M += Nc
  return runs_count[runs_goal]
```